

WORKFLOW ANALYSIS REPORT

Batch Video Audio Enhancer

An n8n Automation for Batch Video Audio Cleanup and Remux

Prepared: August 18, 2026

Platform: n8n | Engine: FFmpeg | Environment: Windows

1. Executive Summary

The “Batch Video Audio Enhancer” is a local n8n automation that walks through every MP4 file in a source folder, pulls out the audio track, cleans it up with FFmpeg audio filters (noise reduction, frequency shaping, and loudness normalization), and reassembles a new video file that pairs the original picture with the improved sound. Temporary files are removed automatically once each video is finished, and the workflow loops through the whole folder in a single manually-triggered run.

The workflow is compact — eleven nodes in total — but it chains together file-system discovery, iterative batch processing, and three sequential FFmpeg invocations per file. It is designed for a specific Windows machine: all paths are hard-coded to the E: drive, and a fixed local path to an FFmpeg 9.0.1 “essentials build” is embedded directly in the command strings.

2. Purpose and Use Case

This workflow solves a common post-production problem: video footage with usable picture but weak, noisy, or unevenly leveled audio (for example, screen recordings, webcam interviews, or field recordings). Rather than opening each clip individually in a video or audio editor, the workflow batch-processes an entire folder unattended, applying the same enhancement chain to every file and producing ready-to-use output copies while leaving the originals untouched.

Typical scenarios this suits well:

- Cleaning up a backlog of recorded lectures, interviews, or webinars before publishing.
- Normalizing loudness across a batch of clips so they play back at a consistent volume.
- Removing steady background hiss or hum (fans, room tone, AC) from voice-heavy recordings.
- Running as a repeatable local pipeline without needing a paid audio-restoration tool.

3. Workflow Architecture

The workflow follows a linear “discover → loop → process → cleanup” pattern, visualized in the diagram on the final page of this report. At a high level:

- A manual trigger starts the run.
- A Set node centralizes the three folder paths the workflow depends on.
- A shell command lists every .mp4 file in the input folder.
- A Code node turns that raw listing into a structured array of items.
- A Split In Batches node loops through the items one at a time, feeding a five-step FFmpeg processing chain, then looping back until the list is exhausted.
- A No-Op node marks completion once the loop's “done” branch fires.

Within each loop iteration, the five processing nodes run strictly in sequence: build the temp audio path, extract audio, enhance audio, remux the video, then delete the temporary WAV files before advancing to the next file. This means the workflow processes files one at a time rather than in parallel — a deliberate trade-off (see Section 6) that keeps disk and CPU load predictable at the cost of overall runtime.

4. Node-by-Node Breakdown

Node	Type	Function
Start (Manual Trigger)	Manual Trigger	Entry point. The workflow is started by hand rather than on a schedule or webhook.
Config (EDIT THESE PATHS)	Set	Defines three workflow-level variables: inputDir, outputDir, and tmpDir, all pointing to folders on the E: drive. This is the single place a user edits before running the workflow.
List Video Files	Execute Command	Runs the Windows command "dir /b" against the input directory, filtered to *.mp4, producing a bare newline-separated list of filenames.
Build File List	Code (JavaScript)	Parses the stdout from the previous step into an array of items, each carrying fileName and a fully-qualified filePath (inputDir + fileName).
Loop Videos	Split In Batches	Iterates over the file list one item at a time, routing each item down the "loop" output and sending a final "done" signal once every item has been processed.
Build Audio Path	Set	For the current item, computes outputPath — the temporary .wav location the raw extracted audio will be written to (tmpDir + fileName with "_raw.wav" appended).
Extract Audio	Execute Command	Runs FFmpeg to strip the audio track out of the source video and save it as a 16-bit PCM WAV file at 44.1 kHz, stereo.
Enhance Audio	Execute Command	Runs FFmpeg again on the raw WAV, applying a noise-reduction, band-pass, and loudness-normalization filter chain, and writes the result to a second WAV file ("_enhanced.wav").
Remux Video (replace audio)	Execute Command	Runs FFmpeg a third time, combining the original video stream (copied, not re-encoded) with the enhanced audio stream (encoded to AAC) into a new output MP4.
Cleanup Temp Files	Execute Command	Deletes the two temporary WAV files (raw and enhanced) created during processing, then loops back to "Loop Videos" to pick up the next file.
Finished	No-Op	Reached once every file in the batch has been processed; marks the end of the run.

5. The FFmpeg Processing Chain

Every file passes through three separate FFmpeg invocations. Each is examined below.

5.1 Step 1 — Extract Audio

Strips the audio out of the source video without re-encoding video (-vn discards video), downmixing/keeping to 2 channels, resampling to 44.1 kHz, and writing uncompressed 16-bit PCM so later filtering has no lossy artifacts to work from.

```
ffmpeg -y -i "<video>" -vn -ac 2 -ar 44100 -c:a pcm_s16le "<raw.wav>"
```

5.2 Step 2 — Enhance Audio

Applies a three-stage audio filter chain to the raw WAV:

- `afftdn=nf=-25` — FFT-based denoiser; `nf` sets the assumed noise floor in dB, subtracting steady background noise (hiss, fan hum, room tone).
- `highpass=f=80 / lowpass=f=12000` — a band-pass filter that removes rumble below 80 Hz and content above 12 kHz, tightening the signal around the speech-relevant range.
- `loudnorm=I=-16:TP=-1.5:LRA=11` — EBU R128 loudness normalization targeting -16 LUFS integrated loudness, a -1.5 dBTP true-peak ceiling, and a loudness range of 11 LU — broadcast-style, consistent-volume output.

```
ffmpeg -y -i "<raw.wav>" -af "afftdn=nf=-25,highpass=f=80,lowpass=f=12000,loudnorm=I=-16:TP=-1.5:LRA=11" -c:a pcm_s16le "<enhanced.wav>"
```

5.3 Step 3 — Remux Video

Recombines the original video stream with the newly enhanced audio. The video stream is copied (-c:v copy), so there is no quality loss and no re-encode time; the enhanced audio is encoded to AAC at 192 kbps for compatibility. -shortest trims the output to the length of the shorter of the two streams.

```
ffmpeg -y -i "<video>" -i "<enhanced.wav>" -map 0:v:0 -map 1:a:0 -c:v copy -c:a aac -b:a 192k -shortest "<output>_enhanced.mp4"
```

5.4 Cleanup

After the remux succeeds, a Windows `del` command removes both the `_raw.wav` and `_enhanced.wav` intermediate files, keeping the temp folder from accumulating clutter across a long batch run.

```
cmd /c del /f /q "<raw.wav>" "<enhanced.wav>"
```

6. Configuration Requirements

Before running, the following must be true on the target machine:

- `n8n` has an “Execute Command” node with access to the local shell (this is a locally-installed / self-hosted `n8n` workflow, not one intended for `n8n Cloud`).
- FFmpeg 9.0.1 (essentials build) is installed at the exact path referenced in each command: `E:\ffmpeg-9.0.1-essentials_build\ffmpeg-9.0.1-essentials_build\bin\ffmpeg.exe`.

- The three folders referenced in the “Config” node exist and are writable: E:\Audio Enhance\input video, \ output video, and \temp audio.
- Input files are .mp4 containers — the file listing step filters specifically on *.mp4.

To repoint the workflow at a different machine or folder structure, a user only needs to edit the “Config (EDIT THESE PATHS)” node — however, the FFmpeg executable path is currently hard-coded separately inside three Execute Command nodes rather than derived from Config, so it must be updated in each of those nodes individually (see Section 7).

7. Observations and Recommendations

A few implementation details are worth flagging for anyone maintaining or extending this workflow:

- **Hard-coded FFmpeg path:** the FFmpeg executable location is repeated in three separate command strings. Centralizing it as a fourth variable in the Config node (e.g. `ffmpegPath`) and referencing it via expression would make the workflow portable to another machine with a single edit.
- **Sequential, single-threaded processing:** files are processed one at a time inside Split In Batches. This is safe and predictable, but on a multi-core machine it leaves performance on the table — FFmpeg’s denoise/normalize pass is CPU-bound and could be parallelized (e.g. 2–3 concurrent items) for large batches.
- **No error handling:** none of the Execute Command nodes have failure branches. If FFmpeg fails on a corrupt or oddly-encoded file (or if a path contains characters that break the quoting), the whole run stops rather than skipping that file and continuing with the rest.
- **Fixed filter parameters:** the noise floor (–25 dB), band-pass corners (80 Hz / 12 kHz), and loudness target (–16 LUFS) are tuned generically for speech. Very noisy source audio or music-heavy content may need different values — currently a change requires editing the raw FFmpeg command text rather than exposing these as Config variables.
- **Output overwrite behavior:** `-y` forces FFmpeg to silently overwrite existing output files, so re-running the workflow on the same input folder will silently replace prior “_enhanced.mp4” results.
- **No logging or per-file report:** since there’s no email/notification/summary step, the only feedback on which files succeeded or failed is the raw execution log inside n8n itself.

8. Data Flow Summary

End-to-end, a single file moves through the pipeline as follows:

- input video\clip.mp4 (source, read-only)
- temp audio\clip_raw.wav (extracted, uncompressed audio)
- temp audio\clip_enhanced.wav (denoised, filtered, loudness-normalized)
- output video\clip_enhanced.mp4 (original video + enhanced audio, AAC 192 kbps)
- temp .wav files deleted — only the source and the final enhanced MP4 remain on disk.

9. Conclusion

This workflow is a well-structured, purpose-built local automation: it correctly separates configuration from logic, uses a standard extract → process → remux → cleanup pattern, and leans on FFmpeg for all the heavy lifting rather than reinventing audio processing inside n8n. Its main limitations — machine-specific hard-coded paths, sequential processing, and the absence of per-file error handling — are typical of a personal-use tool and are straightforward to address if the workflow needs to become more portable or resilient. As-is, it is ready to run on the specific Windows environment it was built for.

Appendix — Workflow Diagram

Visual layout of the workflow as configured in n8n, showing the full node graph and connection order described throughout this report.

